Session 03:

Java 8 (Phần 2)

RIKKEI Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

1. Giới thiệu về Stream API
2. Tính năng Optional
3. Giới thiệu về Datetime API
4. Các tính năng Java 17 thông dụng

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Giới thiệu về Stream API

- **Stream** (luồng) là **một đối tượng mới của Java 8 giúp cho việc thao tác** trên **Collection** và **Array** trở nên **dễ dàng và tối ưu hơn**.
- Một **Stream** đại diện cho **một chuỗi các phần tử hỗ trợ các hoạt động tổng hợp tuần tự** (sequential) và **song song** (parallel).
- Tất cả các **Class** và **Interface** của **Stream API** nằm trong gói **java.util.stream**

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Ví dụ về Stream API

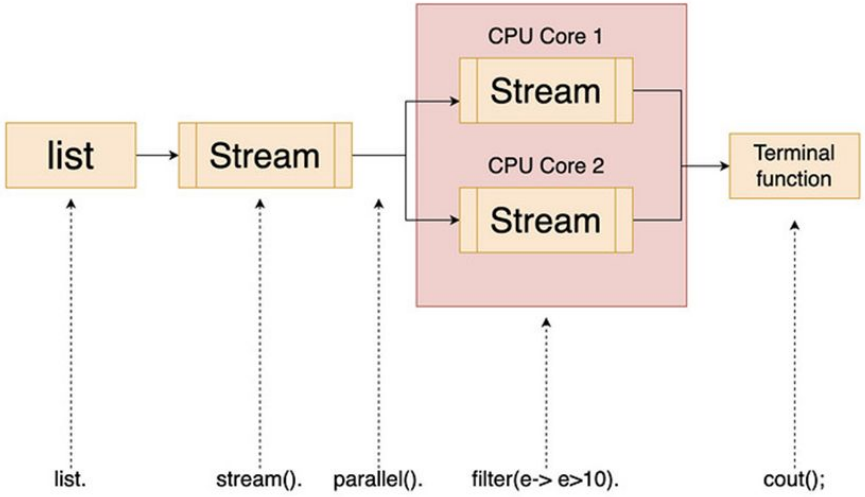
```java
public class StreamExample01 {

    List<Integer> numbers = Arrays.asList(7, 2, 5, 4, 2, 1);

    public void withoutStream() {
        long count = 0;
        for (Integer number : numbers) {
            if (number % 2 == 0) {
                count++;
            }
        }
        System.out.printf("There are %d elements that are even", count);
    }

    public void withStream() {
        long count = numbers.stream().filter(num -> num % 2 == 0).count();
        System.out.printf("There are %d elements that are even", count);
    }

}
```

- **2 phương thức trên ví dụ** đều cho **kết quả giống nhau**, nhưng **Stream ngắn gọn hơn**

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Một số phương thức của Stream API

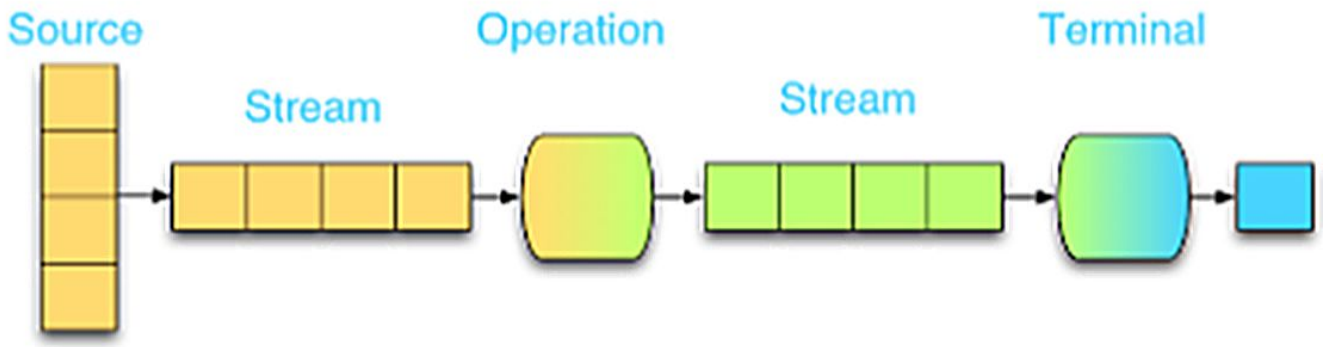
- Trong Java 8, **interface Collection** được hỗ trợ 2 bởi 2 phương thức để tạo Stream:
  - **stream()**: trả về 1 stream sẽ được xử lý theo tuần tự
  - **parallelStream()**: trả về 1 stream song song, các xử lý sau đó sẽ thực hiện song song



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Đặc điểm của Stream API

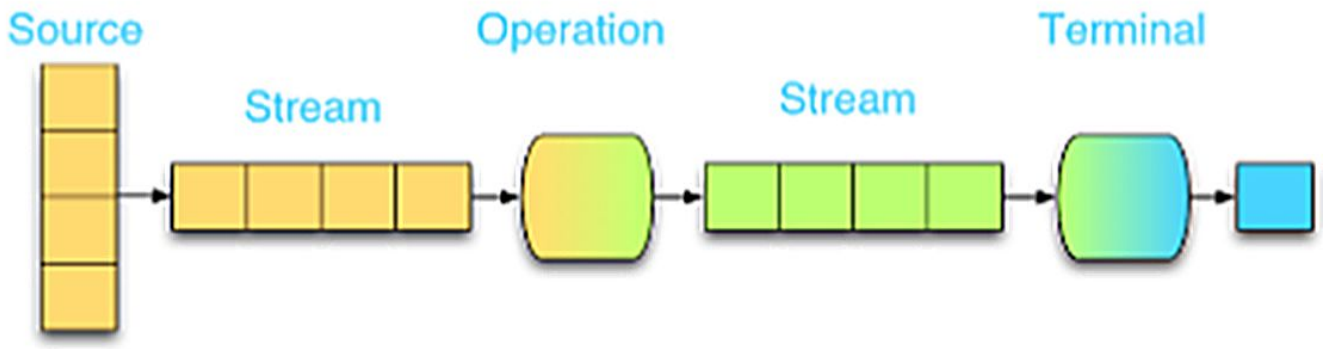
- **Stream** không lưu trữ các phần tử của **Collection** hay **Array**
- **Stream** chỉ thực hiện các phép toán tổng hợp (ví dụ: **filter(), count(),**...)
- **Stream** là **immutable object**. Các hoạt động tổng hợp mà chúng ta thực hiện trên **Collection**, **Array** không làm thay đổi dữ liệu nguồn, chỉ trả lại stream mới.



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Đặc điểm của Stream API

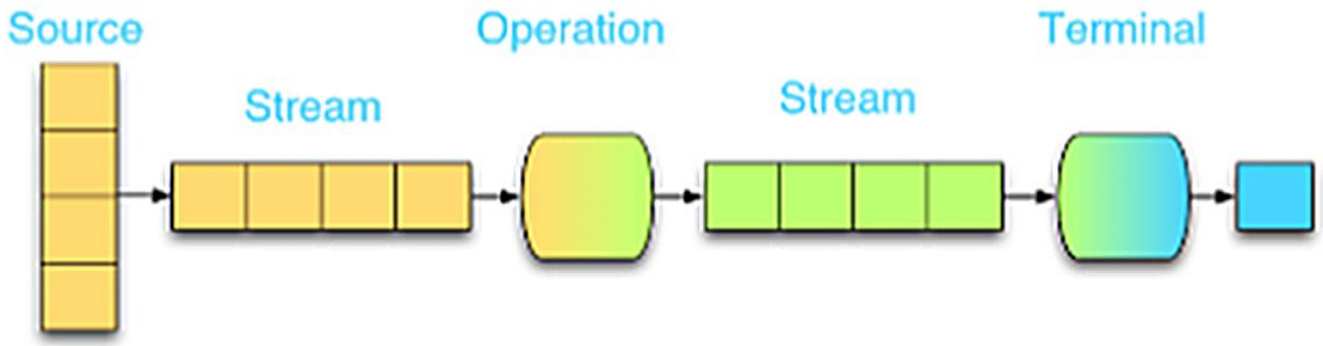
- **Tất cả các hoạt động của Stream** là **lazy**, nghĩa là chúng không được thực hiện cho đến khi cần thiết.
- **Để hiện thực cơ chế lazy,** các thao tác với Stream đều return lại một Stream mới.
- Thao tác tạo **Stream** mới liên tiếp tạo một **pipeline** (tập hợp một loạt các thao tác cần thực thi của Stream)



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Đặc điểm của Stream API

- **Các phần tử của luồng chỉ được truy cập một lần trong suốt vòng đời của Stream**
- **Stream** không thể dùng **index** để truy xuất các phần tử trong **Stream**.
- **Stream** **hỗ trợ thao tác song song** các phần tử trong **Collection** và **Array**.



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

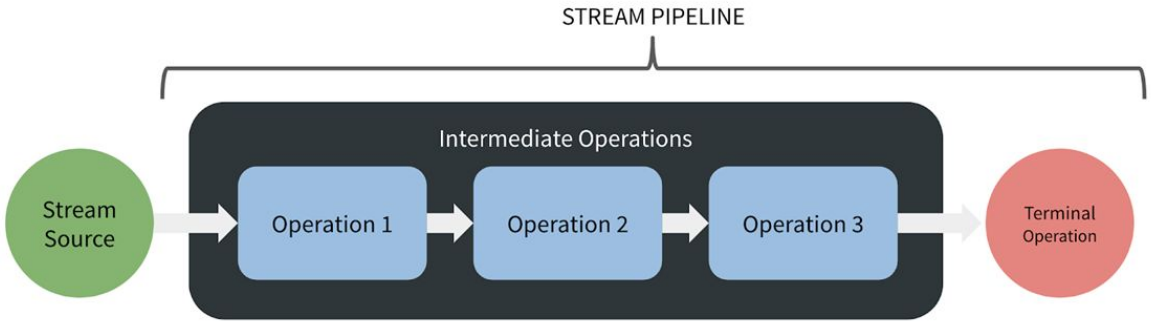
1. So sánh Stream API với Collection

Stream	Collection
- Không phải là một cấu trúc dữ liệu - Stream là một luồng thực hiện tính toán các phần tử theo yêu cầu	- Collection là một cấu trúc dữ liệu chứa các phần tử trong bộ nhớ - Các phần tử trong Collection sẽ được tính toán trước khi được thêm vào Collection
Các Stream có yếu tố tính lười biếng (lazy)	Các Collection có các yếu tố tính tức thời (eager)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Stream chia 3 giai đoạn làm việc

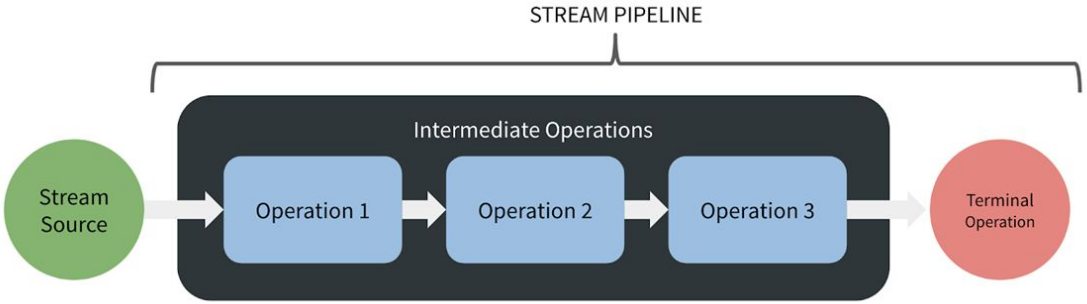
- **Tạo Stream** (**Stream Source**)
- **Thực hiện các thao tác trung gian** (**Intermediate Operations**) trên stream ban đầu để chuyển đổi nó thành một stream khác và tiếp tục thực hiện các hoạt động trung gian khác (Có thể có nhiều hoạt động trung gian)
- **Thực hiện thao tác đầu cuối** (**Terminate Operation**) trên stream cuối cùng để nhận kết quả và sau đó không thể sử dụng lại chúng



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Các cách tạo Stream thường gặp

- **Tạo Stream cho kiểu nguyên thủy** (**IntStream, LongStream, ...**)
- **Tạo Stream từ cấu trúc dữ liệu khác như**:
  - Arrays
  - Collection
  - Stream.generate()
  - Stream.iterate()
  - Regex APIs



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

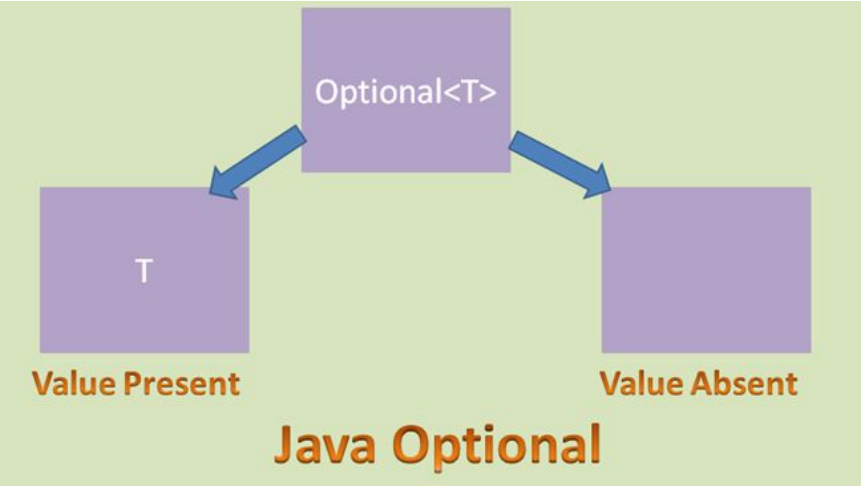
1. Các Stream API thông dụng

- **Phương thức filter():** giúp loại bỏ các phần tử dựa trên các tiêu chí nhất định
- **Phương thức map():** giúp chuyển đổi dữ liệu các phần tử tới các kết quả tương ứng
- **Phương thức sorted():** giúp sắp xếp các phần tử theo một thứ tự xác định
- **Phương thức forEach():** giúp duyệt qua các phần tử của Stream
- **Phương thức collect():** giúp thu thập kết quả Stream sang một Collection
- **Phương thức count():** trả về tổng số phần tử cho dữ liệu luồng.
- **Phương thức parallelStream():** giống stream() nhưng sử dụng trong môi trường **Multi-Thread** để tăng tốc hiệu suất chương trình..

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Tính năng Optional

- **Lớp Optional<T> trong Java 8** được sử dụng để kiểm tra một biến có tồn tại hay không
- **Ưu điểm:** tránh lỗi **NullPointerException** (NPE) lúc runtime
- **Giống như Collection và Array, Optional** cũng là một vùng chứa nhiều giá trị ở bất kỳ kiểu dữ liệu T nào



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Khởi tạo đối tượng Optional<T>

- **Tạo một đối tượng Optional rỗng (empty)**

```
Optional<String> optional = Optional.empty();
```

- **Tạo một đối tượng Optional chứa giá trị non-null**

```
String str = “javacoder”;
Optional<String> optional = Optional.of(str);
```

- **Tạo một đối tượng Optional chứa giá trị cụ thể hoặc giá trị rỗng (empty) nếu tham số là null**

```
Optional<String> optional = Optional.ofNullable(obj);
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Các phương thức với Optional<T>

- **Phương thức isPresent():** giúp kiểm tra tồn tại giá trị hay không trước khi sử dụng
- **Phương thức get():** giúp lấy giá trị của đối tượng ra
- **Phương thức ifPresent():** giúp kiểm tra nếu tồn tại giá trị thì thực thi gì đấy
- **Phương thức orElse():** giúp trả về giá trị mặc định nếu giá trị của đối tượng Optional rỗng
- **Phương thức orElseGet():** tương tự orElse() nhưng xử lý logic phức tạp hơn (perf tốt hơn)
- **Phương thức orElseThrow()**: giúp ném ra Exception nếu đối tượng Optional bị lỗi
- **Phương thức flatMap():** giúp gộp lại các Optional lồng nhau thành một Optional duy nhất

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Giới thiệu về Datetime API

- **So sánh Datetime API kiểu cũ và mới:**

Nhược điểm của java.util.Date & Calendar	Bước tiến của java.time (JSR 310)
Mutable (dễ bị thay đổi): giá trị ngày tháng có thể bị sửa đổi ngoài ý muốn	Immutable (bất biến): giá trị ngày tháng KHÔNG thể bị sửa đổi ngoài ý muốn
Thread-unsafe: Gây lỗi dữ liệu trong môi trường đa luồng	Thread-safe: Phù hợp với ứng dụng đa luồng hiện đại

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Các lớp cốt lõi trong java.time

Giờ địa phương (Không gắn múi giờ)

- LocalDate (Chỉ lưu ngày/tháng/năm)
- LocalTime (Chỉ lưu giờ/phút/giây)
- LocalDateTime (Kết hợp Ngày + Giờ)

Giờ toàn cầu (Gắn với múi giờ)

- ZonedDateTime
- OffsetDateTime

Ứng dụng: Xây dựng hệ thống Global, lưu trữ chuẩn xác xuống Database

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Đo lường khoảng thời gian & Định dạng

RIKKEI Academy
where the dream begins

Đo lường khoảng cách:

Period: Khoảng cách thời gian đo bằng Ngày / Tháng / Năm

Duration: Khoảng cách đo bằng Giờ / Phút / Giây / Nano giây.

Định dạng chuẩn (Formatting):

Sử dụng **DateTimeFormatter** (Thread-safe, thay thế hoàn toàn cho **SimpleDateFormat**)

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Đo lường khoảng thời gian & Định dạng

```
// Lấy thời gian hiện tại
LocalDate today = LocalDate.now();

// Tính khoảng cách giữa 2 ngày (Period)
LocalDate futureDate = LocalDate.of(2030, 1, 1);
Period period = Period.between(today, futureDate);

// Định dạng chuỗi ngày tháng
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String formattedDate = today.format(formatter);
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Các tính năng Java 17 thông dụng

- **Bản ghi (Records):**
  - Tối giản hóa việc tạo các class chỉ để chứa các dữ liệu (**DTO - Data Transfer Object**)
  - Tự động sinh ra **constructor, getters, equals(), hashCode(), toString()**

Cú pháp một dòng duy nhất:

```
public record StudentDto(String name, double score) {}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Các tính năng Java 17 thông dụng

- **Xử lý chuỗi đa dòng với TextBlocks:**
  - Sử dụng cú pháp **“”” ** để định nghĩa khối văn bản.
  - **Ứng dụng**: Nhúng câu lệnh SQL, mã HTML, hoặc JSON trực tiếp vào code Java.
  - Loại bỏ hoàn toàn việc cộng chuỗi rườm rà (concatenation) và ký tự escape (\n, \”)

```
String jsonResponse = """
{
    "status": "success",
    "message": "Welcome to Java 17"
}
""";
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Các tính năng Java 17 thông dụng

- **Pattern Matching cho instanceof:**
  - Loại bỏ việc phải ép kiểu (casting) thủ công ngay sau khi kiểm tra kiểu dữ liệu..
  - Giúp mã nguồn ngắn gọn, liền mạch và tránh rủi ro **ClassCastException**

Trước đây:

```java
if (obj instanceof String) {
    String s = (String) obj; // Ép kiểu thủ công
    System.out.println(s.toLowerCase());
}
```

Java 17:

```java
if (obj instanceof String s) {
    System.out.println(s.toLowerCase());
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- Suy nghĩ theo hướng khai báo (Declarative pipeline), tập trung vào kết quả mong muốn hơn là các bước thực hiện chi tiết, sử dụng Streams và Lambdas.
- Tự bảo vệ mã nguồn, nói không với NPE (NullPointerException) bằng cách sử dụng kiểu Optional để xử lý các giá trị có thể null một cách an toàn.
- Ưu tiên Bất biến (Immutability) để đảm bảo an toàn đa luồng (Thread-safety), ví dụ như sử dụng Datetime API mới.
- Tối giản hóa cấu trúc với cú pháp thế hệ mới (Java 17+), áp dụng các tính năng như Records, Pattern Matching để code ngắn gọn và dễ đọc hơn.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN